

Desenvolvimento de um Autômato Celular Intitulado “Jogo da Vida” por John Conway na Linguagem Python

Thiago Marques Reis¹

¹Departamento de Tecnologia – Universidade Estadual de Feira de Santana (UEFS) –
Feira de Santana – BA – Brasil

theicefrosts@gmail.com

1. Introdução

O objetivo desse problema é desenvolver de forma autônoma um autômato celular criado pelo matemático John Conway em 1970, intitulado “Jogo da Vida”. O jogo foi criado originalmente para reproduzir as alterações e mudanças em grupos de seres vivos e até hoje é utilizado em diversas áreas da ciência, ou seja, a motivação para esse trabalho de reprodução, de autoria própria, não mudou. Entretanto, através das percepções geradas durante a reconstrução do jogo, é compreensível que o mesmo pode influenciar a criação de novas simulações que serão utilizadas para outros fins, desde a simulação espacial de fenômenos com comportamentos caóticos até o combate ao câncer.

A solução do problema consiste em um projeto baseado no conceito de modularização, capaz de prover um jogo de terminal que representa uma sociedade em um tabuleiro 10×10 , com células vivas e mortas que interagem entre si, sendo modificadas de acordo com os vizinhos ao seu redor. O jogo é uma simulação, que originalmente é automatizada, mas nessa reprodução ela é controlada pelo usuário, que pode também escolher se o jogo vai ser manual (células vivas são selecionadas pelo usuário) ou aleatório (células vivas são escolhidas aleatoriamente).

Em uma breve descrição, foram utilizados como recursos algorítmicos entradas para o usuário controlar a continuidade do jogo e escolher o modo de jogo e saídas para o usuário saber o que está acontecendo durante todo o jogo. Os conceitos de modularização, estruturas condicionais, estruturas de repetição, listas, bibliotecas e tratamento de dados foram também muito utilizados para recriar as funcionalidades do jogo e para criar algumas funcionalidades extras.

2. Metodologia

Neste trabalho, houveram decisões individuais e coletivas. Entre as principais decisões individuais estão a representação de células vivas com um “V” e de células mortas com um “.”, a utilização das bibliotecas “time” e “os”, o uso de constantes e a criação de uma função que limpa a tela do terminal. Já entre as principais decisões coletivas estão o uso de saídas bem otimizadas, o controle da continuidade do jogo pelo usuário, o uso da biblioteca “random” para o “modo aleatório” do jogo, a finalização do jogo quando o jogo atinge a simetria ou a “vida parada”, a escolha individual dos símbolos utilizados e a utilização de matrizes.

2.1. Requisitos

Entre as exigências que foram impostas para o desenvolvimento do “Jogo da Vida” estão:

- O “Jogo da Vida” deverá representar uma sociedade em um tabuleiro 10×10 .
- Devem ser definidos os símbolos que representarão as células mortas e vivas.
- No início do jogo, o usuário poderá inserir um padrão inicial, determinando as células vivas.
- Cada célula da matriz pode estar somente em dois estados: viva ou morta.
- Os vizinhos são as 8 células ao redor: cima, baixo, esquerda, direita e diagonais.
- A cada iteração, o estado da matriz muda do seguinte modo:
 1. Solidão: uma célula viva com menos de 2 vizinhos vivos morre.
 2. Sobrevivência: uma célula viva com 2 ou 3 vizinhos vivos sobrevive.
 3. Superpopulação: uma célula viva com mais de 3 vizinhos vivos morre.
 4. Reprodução: uma célula morta com 3 vizinhos vivos se torna viva.
- Se todas as células do tabuleiro estiverem mortas, sua simulação deve chegar ao fim.
- As principais tarefas do software devem ser implementadas utilizando o conceito de modularização.

2.2. Descrição do Algoritmo

```
1 # Bibliotecas importadas.
2 import time # Importa a biblioteca time, que não é necessária nesse caso (extra), mas ela permite o controle do fluxo do código.
3 import os # Importa a biblioteca os, que também não é necessária nesse caso (extra), mas ela permite a interação com o sistema operacional.
4 import random # Importa a biblioteca random, necessária para formação aleatória do tabuleiro, que permite trabalhar com dados ou valores aleatórios.
5
6 # Atribuição de variáveis. Por convenção, em muitas linguagens, o nome com letras maiúsculas indica que o mesmo é uma constante.
7 # Nesse caso, utilizei para dizer que o tamanho do tabuleiro sempre vai ser 10 X 10 e para a representação de vivo e morto.
8 TAMANHO = 10
9 CELULA_VIVA = True
10 CELULA_MORTA = False
11 SIMBOLO_CELULA_VIVA = 'V'
12 SIMBOLO_CELULA_MORTA = '.'
```

Figura 1. Importação de bibliotecas e atribuição de constantes.

Inicialmente, são importadas 3 bibliotecas (random, os e time), onde duas são para funcionalidades extras (os e time). A biblioteca “random” é necessária para a geração de padrões aleatórios no tabuleiro (matriz). A “os” permite a interação com o sistema operacional, permitindo criar recursos que foram utilizados, como o de limpar a tela. Por fim, a “time” permite controlar o fluxo do código, permitindo que o usuário tenha uma experiência agradável, ao invés de um código com possíveis “delays”. Além disso, há a atribuição de constantes, que são o tamanho do tabuleiro e a representação, tanto “interna” quanto “externa” das células vivas e mortas.

```

14 # Cria uma função para limpar a tela.
15 def limpar_tela():
16     # Essa função do módulo permite que os comandos sejam executados diretamente no terminal. Nesse caso, há uma estrutura condicional onde são
17     # executados diferentes comandos para limpar a tela, dependendo do sistema operacional (Windows, Linux/macOS).
18     os.system('cls' if os.name == 'nt' else 'clear')

```

Figura 2. Criação de uma função para limpar a tela.

Nessa figura, a primeira função do código é criada. Nela, a função “system” do módulo “os” é chamada, o que detecta qual o sistema operacional do usuário. Dentro, há uma condicional, onde se o sistema for Windows, um comando de limpar a tela é executado e se ele for Linux/MacOs, outro comando de limpar a tela é executado.

```

20 # Cria uma função que cria um tabuleiro somente com células mortas e recebe
21 # como parâmetros "tamanho", que é o tamanho ou quantidade de linhas e de colunas.
22 def criar_tabuleiro_vazio(tamanho):
23
24     # Atribuição de variável. É atribuída uma lista vazia, que armazenará o tabuleiro (matriz).
25     tabuleiro_completo = []
26     # Laço de repetição(for) que percorre todas as linhas da matriz ou tabuleiro.
27     for linha in range(tamanho):
28         # Atribuição de variável. É atribuída uma lista vazia, que armazenará as linhas do tabuleiro (matriz).
29         linha_exibida = []
30         # Laço de repetição(for) que percorre todos os elementos da linha.
31         for celula in range(tamanho):
32             # Adiciona CELULA_MORTA ('.') a variável "linha_exibida", ou seja, em cada elemento.
33             linha_exibida.append(CELULA_MORTA)
34             # Adiciona a variável "linha_exibida" a variável "tabuleiro_completo".
35             tabuleiro_completo.append(linha_exibida)
36
37     # Retorna a variável "tabuleiro_completo" ao final da função.
38     return tabuleiro_completo

```

Figura 3. Criação de uma função para criar um tabuleiro vazio (células mortas).

É criada uma função que cria um tabuleiro vazio e recebe como parâmetro “tamanho”. É montada uma matriz (lista de listas) baseada no parâmetro tamanho. Dentro dos laços de repetição que percorrem todo o tabuleiro, no primeiro há uma lista vazia que vai armazenar cada linha e no segundo é adicionado em cada célula dessa matriz o valor de CELULA_MORTA e depois essas linhas são armazenadas na lista inicialmente vazia chamada “tabuleiro_completo”. Por fim, a função retorna “tabuleiro_completo”, que possui um tabuleiro de células mortas.

```

40 # Cria uma função que exibe o estado do tabuleiro e recebe como parâmetros o "estado", que é o tabuleiro ou matriz.
41 def exibir_estado(estado):
42
43     # Chama a função que limpa o terminal.
44     limpar_tela()
45     # Imprime o nome do jogo na tela.
46     print("--- Jogo da Vida ---")
47     # Laço de repetição(for) que percorre todas as linhas da matriz ou tabuleiro.
48     for linha in estado:
49         # Atribuição de variável. É atribuída uma lista vazia, que armazenará as linhas do tabuleiro (matriz).
50         linha_para_exibir = []
51         # Laço de repetição(for) que percorre todos os elementos da linha.
52         for celula in linha:
53             # Estrutura condicional, onde se a variável "celula" (elemento da linha) for igual a constante "CELULA_VIVA" (True),
54             # é adicionada a constante "SIMBOLO_CELULA_VIVA"(V) a variável "linha_para_exibir".
55             # Caso não, é adicionada a constante "SIMBOLO_CELULA_MORTA"(".") a variável "linha_para_exibir".
56             if celula == CELULA_VIVA:
57                 linha_para_exibir.append(SIMBOLO_CELULA_VIVA)
58             else:
59                 linha_para_exibir.append(SIMBOLO_CELULA_MORTA)
60             # Imprime na tela a variável "linha_para_exibir" com um espaço.
61             print(" ".join(linha_para_exibir))
62         # Imprime uma barreira de "-" para mostrar onde o tabuleiro termina.
63         print("-" * (TAMANHO * 2 - 1))

```

Figura 4. Criação de uma função para a exibição de um tabuleiro .

É criada uma função que exibe o “estado do tabuleiro” e recebe como parâmetros “estado”, que é justamente a matriz a ser exibida. A função que limpa a tela é evocada e depois dois loops “for” percorrem a matriz assim como no exemplo anterior, mas desta vez percorrendo diretamente (ao invés do range). Para cada elemento percorrido, uma estrutura condicional verifica se a célula é viva ou morta e adiciona o respectivo símbolo. Cada linha armazenada é alterada pelo “join”, que separa os elementos da lista com espaços e uma barreira de linhas é impressa ao final.

```

65 # Cria uma função que calcula a quantidade de vizinhos vivos em uma posição específica e recebe como parâmetros "estado, linha, coluna".
66 def contar_vizinhos_vivos(estado, linha, coluna):
67
68     # Atribuição de variável. Começa um "contador" da variável vizinhos_vivos igualando a mesma a 0.
69     vizinhos_vivos = 0
70     # Atribuição de variável. A funcionalidade "len" calcula a quantidade de linhas (listas) em "estado" (tabuleiro ou matriz).
71     num_linhas = len(estado)
72     # Atribuição de variável. A funcionalidade "len" calcula a quantidade de colunas em "estado" na posição 0.
73     num_colunas = len(estado[0])
74
75     # Laço de repetição(for) que percorre os 8 vizinhos possíveis.
76     for i in range(max(0, linha - 1), min(num_linhas, linha + 2)):
77         for j in range(max(0, coluna - 1), min(num_colunas, coluna + 2)):
78             # Estrutura condicional que evita que a própria célula seja contada.
79             if i == linha and j == coluna:
80                 continue
81             # Estrutura condicional que verifica se o vizinho está vivo.
82             if estado[i][j] == CELULA_VIVA:
83                 # Adiciona +1 a variável "vizinhos_vivos" ("contador").
84                 vizinhos_vivos += 1
85
86     # Retorna a quantidade guardada na variável "vizinhos_vivos" ao final da função.
87     return vizinhos_vivos

```

Figura 5. Criação de uma função para calcular a quantidade de vizinhos vivos .

Cria uma função que calcula quantos vizinhos vivos uma determinada célula tem e recebe como parâmetros “linha, coluna e estado”. Logo em seguida, um “contador de vizinhos” é atribuído como 0 e o número de linhas e colunas é calculado pela função

“len”. Um laço de repetição começa, utilizando as funções “max e min” para limitar a quantidade de vizinhos somente aos que estão no tabuleiro e pula a célula determinada com a condicional e a funcionalidade “continue”. Se os vizinhos forem células vivas, o contador é alterado com +1 e a função retorna justamente o número de vizinhos vivos.

```
89 # Cria uma função que "calcula" o próximo estado do tabuleiro e recebe como parâmetros "estado_atual", ou seja, o tabuleiro atual.
90 def calcular_proximo_estado(estado_atual):
91
92     # Atribuição de variável. A funcionalidade "len" calcula a quantidade de linhas (listas) em "estado_atual" (tabuleiro ou matriz).
93     num_linhas = len(estado_atual)
94     # Atribuição de variável. A funcionalidade "len" calcula a quantidade de colunas em "estado_atual" na posição 0.
95     num_colunas = len(estado_atual[0])
96     # Atribuição de variável. Começa o próximo estado com um tabuleiro morto.
97     proximo_estado = criar_tabuleiro_vazio(num_linhas)
98
99     # Laço de repetição(for) que percorre todas as linhas da matriz ou tabuleiro.
100    for i in range(num_linhas):
101        # Laço de repetição(for) que percorre todas as colunas da matriz ou tabuleiro.
102        for j in range(num_colunas):
103            # Atribuição de variável. Conta os vizinhos vivos na matriz.
104            vizinhos = contar_vizinhos_vivos(estado_atual, i, j)
105            # Atribuição de variável. Observa se a célula que está sendo percorrida naquele momento é viva.
106            celula_atual_viva = estado_atual[i][j] == CELULA_VIVA
107
108            # Estrutura condicional que aplica as regras do Jogo da Vida:
109
110            # 1. Solidão: Célula viva com menos de 2 vizinhos vivos morre.
111            if celula_atual_viva and vizinhos < 2:
112                proximo_estado[i][j] = CELULA_MORTA
113            # 2. Sobrevivência: Célula viva com 2 ou 3 vizinhos vivos sobrevive.
114            elif celula_atual_viva and (vizinhos == 2 or vizinhos == 3):
115                proximo_estado[i][j] = CELULA_VIVA
116            # 3. Superpopulação: Célula viva com mais de 3 vizinhos vivos morre.
117            elif celula_atual_viva and vizinhos > 3:
118                proximo_estado[i][j] = CELULA_MORTA
119            # 4. Reprodução: Célula morta com exatamente 3 vizinhos vivos torna-se viva.
120            elif not celula_atual_viva and vizinhos == 3:
121                proximo_estado[i][j] = CELULA_VIVA
122            # Caso contrário (célula morta sem 3 vizinhos), continua morta.
123
124    # Retorna o próximo tabuleiro depois de modificar as células.
125    return proximo_estado
```

Figura 6. Criação de uma função para calcular o próximo estado do tabuleiro.

Cria uma função que calcula o próximo estado e recebe como parâmetros o “estado_atual”. O número de linhas e colunas do “estado_atual” é quantificado pela função “len” novamente e a variável próximo estado evoca a função de criar um tabuleiro vazio que vai guardar os resultados. Dentro de dois loops “for” novamente para percorrer toda a matriz, cada célula tem seus vizinhos contados graças a função evocada de contar vizinhos e é observado se a célula atual é viva. Para cada uma, uma condicional é gerada para cada e verifica como a célula vai ficar no próximo tabuleiro seguindo os critérios do jogo. A função retorna o próximo tabuleiro atualizado.

```

127 # Cria uma função que permite ao usuário definir o estado (matriz) inicial manualmente e recebe como parâmetros "tamanho".
128 def inicializar_manual(tamanho):
129     # Atribuição de variável. Controla o loop while.
130     continuar = True
131     # Atribuição de variável. Começa o estado com um tabuleiro morto.
132     estado = criar_tabuleiro_vazio(tamanho)
133     # Imprime uma frase.
134     print("\n--- Modo Manual ---")
135     # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
136     time.sleep(1.5)
137
138     # Estrutura de repetição(while) que continua até o usuário começar a simulação('fim').
139     while continuar:
140         # Evoca uma função, que mostra o estado atual durante a inserção.
141         exibir_estado(estado)
142         # Bloco try que pula para o except quando a entrada estiver errada.
143         try:
144             # Imprime as instruções de como inserir e finalizar no modo manual.
145             print(f"Digite as coordenadas (linha coluna) das células vivas (0 a {tamanho-1}).")
146             print("Exemplo: '3 4' para ativar a célula na linha 3, coluna 4.")
147             print("Digite 'fim' quando terminar.")
148             # Atribuição de variável. É a entrada onde o usuário pode digitar a coordenada ou iniciar a simulação.
149             # As funcionalidades ".strip().lower()" são utilizadas respectivamente para
150             # limpar espaços em branco nas bordas e para converter tudo para minúsculas.
151             entrada = input("Coordenada (linha coluna) ou 'fim': ").strip().lower()
152             # Estrutura condicional onde se "entrada" que o usuário digitou foi "fim", o loop termina e a simulação começa.
153             if entrada == 'fim':
154                 continuar = False
155             # Divide a string de "entrada" com base em espaços em branco, para verificar se ele digitou 2 strings.
156             # Pega tudo isso e coloca em uma lista.
157             dividir_entrada = entrada.split()
158             # Estrutura condicional onde se a quantidade de elementos da lista de "dividir_entrada" for diferente de 2, a entrada é invalidada.
159             if len(dividir_entrada) != 2:
160                 # Um erro é forçado porque a entrada não foi escrita no formato correto ('linha coluna').
161                 raise ValueError("Entrada inválida. Use o formato 'linha coluna'.")
162
163             # Atribuição de variáveis. Tenta transformar as strings da coordenada em um número inteiro.
164             linha = int(dividir_entrada[0])
165             coluna = int(dividir_entrada[1])
166
167             # Estrutura condicional que verifica se o tamanho da linha e da coluna digitados são válidos.
168             if 0 <= linha < tamanho and 0 <= coluna < tamanho:
169                 # Transforma o estado da célula quando a coordenada é digitada, pois estamos usando o True e o False.
170                 estado[linha][coluna] = not estado[linha][coluna]

```

```

171
172     # Atribuição de variável para verificar o estado da célula.
173     esta_viva = estado[linha][coluna]
174     # Atribuição de variável que guarda uma palavra.
175     string_estado = ""
176     # Estrutura condicional que guarda palavras diferentes dependendo se "esta_viva" é True ou False.
177     if esta_viva:
178         string_estado = "Viva"
179     else:
180         string_estado = "Morta"
181
182     # Imprime uma frase dizendo o que aconteceu com o estado da célula.
183     print(f"Célula ({linha},{coluna}) agora {string_estado}.")
184     # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
185     time.sleep(1)
186
187     else:
188         # Imprime mensagem para coordenadas inválidas.
189         print(f"Coordenadas fora do tabuleiro (0 a {tamanho-1}). Tente novamente.")
190         # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
191         time.sleep(1)
192
193     except ValueError as erro:
194         # Imprime mensagem de erro para entrada inválida.
195         print(f"Erro: {erro}. Tente novamente.")
196         # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
197         time.sleep(1.5)
198     except Exception as erro:
199         # Imprime mensagem para erros inesperados.
200         print(f"Ocorreu um erro inesperado: {erro}")
201         # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
202         time.sleep(1.5)
203
204     # Retorna o estado final definido pelo usuário.
205     return estado

```

Figuras 7 e 8. Criação de uma função para inicialização do modo manual.

Cria uma função que inicializa o modo manual e recebe como parâmetros “tamanho”. Ela começa gerando um “estado” totalmente morto. Logo depois, entra num laço de repetição (while continuar). Dentro do loop, primeiro é exibido o tabuleiro atual e as instruções necessárias, com uma pequena pausa. Depois, pede ao usuário para digitar as coordenadas ou 'fim'. Um bloco “try...except” faz o tratamento de entradas inválidas, mostrando o erro. Se o usuário digita 'fim', o ciclo é encerrado. Se as coordenadas forem válidas, o estado da célula naquela posição é invertido (not): viva se torna morta e vice-versa. Uma mensagem de feedback é mostrada com outra pequena pausa. Coordenadas fora do tabuleiro geram apenas um aviso. Ao sair do loop, a função retorna o estado configurado manualmente.

```
206 # Cria uma função que gera um tabuleiro de forma aleatória e recebe como parâmetros tamanho (tamanho do tabuleiro) e
207 # probabilidade (chance da célula se tornar viva).
208 def inicializar_aleatorio(tamanho, probabilidade = 0.3):
209
210     # Imprime uma frase.
211     print("\n--- Modo Aleatório ---")
212     # Atribuição de variável. Cria um tabuleiro vazio.
213     estado = criar_tabuleiro_vazio(tamanho)
214     # Laço de repetição(for) que percorre todas as linhas da matriz ou tabuleiro.
215     for i in range(tamanho):
216         # Laço de repetição(for) que percorre todas as colunas da matriz ou tabuleiro.
217         for j in range(tamanho):
218             # Evoca a funcionalidade random() do módulo random, onde o "< probabilidade" define a probabilidade da célula estar viva
219             if random.random() < probabilidade :
220                 # Define que a célula percorrida é viva.
221                 estado[i][j] = CELULA_VIVA
222     # Imprime uma frase.
223     print("Tabuleiro inicial aleatório gerado.")
224     # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
225     time.sleep(2)
226
227     # Retorna o estado (tabuleiro ou matriz).
228     return estado
229
```

Figura 9. Criação de uma função para inicialização do modo aleatório.

Cria uma função chamada “inicializar_aleatorio” que recebe como parâmetros “tamanho, probabilidade = 0.3” e gera um tabuleiro usando padrões aleatórios com funcionalidades da biblioteca “random”. Após criar um estado inicial vazio, ela percorre cada célula (i, j). Para cada célula, gera um número aleatório entre 0 e 1 (random.random()). Se esse número for menor que a probabilidade definida(0.3), a célula se torna uma CELULA_VIVA. Dessa forma, a probabilidade controla a "probabilidade" de células vivas iniciais. Concluída a população aleatória, uma mensagem confirma a geração, há uma pequena pausa com uma funcionalidade da biblioteca “time”, e o estado resultante é retornado.

```

230 # Cria uma função que verifica se o jogo deve terminar (células mortas ou "vida parada") e recebe como
231 # parâmetros "estado_atual" e "estado_anterior", que é basicamente o tabuleiro atual e o anterior.
232 def verificar_fim_jogo(estado_atual, estado_anterior):
233     # Atribuição de variável. Suposição que o jogo acabou.
234     todas_mortas = True
235     # Laço de repetição(for) que percorre todas as linhas da matriz ou tabuleiro.
236     for linha in estado_atual:
237         # Laço de repetição(for) que percorre todas as colunas da matriz ou tabuleiro.
238         for celula in linha:
239             # Estrutura condicional onde se houver qualquer célula viva, o jogo não termina.
240             if celula == CELULA_VIVA:
241                 # Nega a suposição.
242                 todas_mortas = False
243
244     # Estrutura condicional caso o jogo tenha acabado (todas as células mortas).
245     if todas_mortas:
246         # Imprime uma frase.
247         print("\nFim do jogo: Todas as células morreram.")
248         # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
249         time.sleep(2)
250
251         # Indica que o jogo deve acabar.
252         return True
253
254     # Verifica se o estado atual é igual ao estado anterior (vida parada).
255     if estado_atual == estado_anterior:
256         # Imprime uma frase.
257         print("\nFim do jogo: O estado tornou-se estável (vida parada).")
258         # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
259         time.sleep(2)
260
261         # Indica que o jogo deve acabar
262         return True
263
264     # Indica que o jogo deve continuar.
265     return False

```

Figura 10. Criação de uma função para verificar se é fim de jogo.

Cria uma função que verifica se é fim de jogo e recebe como parâmetros. Ela realiza duas verificações principais. Primeiro, procura pela ocasião em que todas as células estão mortas em “estado_atual”, procurando por alguma CELULA_VIVA. Se não encontrar nenhuma após verificar tudo, conclui que todas morreram, imprime uma mensagem, pausa e retorna True (fim do jogo). Se houver células vivas, ela passa para a segunda verificação: compara o “estado_atual” com o “estado_anterior”. Se forem idênticos, significa que a ocasião é de “vida parada”, onde ela imprime a mensagem de estabilidade, pausa e retorna True. Se nenhuma dessas condições for verdadeira, o jogo ainda está em andamento e a função retorna False, já que estamos utilizando um sistema booleano.

```

267 # Cria uma função principal do "Jogo da Vida".
268 def main():
269
270     # Atribuição de variável. Vai guardar futuramente o tabuleiro da rodada anterior, por isso None.
271     estado_anterior = None
272     # Atribuição de variável. Vai guardar futuramente o tabuleiro daquele momento, por isso None.
273     estado_atual = None
274     # Atribuição de variável. Vai acumular as gerações passadas.
275     geracao = 0
276
277     # Atribuição de variável para controlar o loop (while).
278     modo_valido = True
279
280     # Laço de repetição while que continua até "modo_valido" ser False.
281     while modo_valido:
282         # Entrada de dados. O usuário escolhe o modo de jogo.
283         modo = input("Escolha o modo de início ('manual' ou 'aleatorio'): ").strip().lower()
284         # Estrutura condicional que verifica o modo escolhido e evoca a respectiva função. O loop termina quando um modo válido for escolhido.
285         if modo == 'manual':
286             estado_atual = inicializar_manual(TAMANH0)
287             # Interrompe o loop.
288             modo_valido = False
289         elif modo == 'aleatorio':
290             estado_atual = inicializar_aleatorio(TAMANH0)
291             # Interrompe o loop.
292             modo_valido = False
293         else:
294             # Imprime uma frase.
295             print("Modo inválido. Tente novamente.")
296             # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
297             time.sleep(1)
298
299     # Atribuição de variável para controlar o loop (while).
300     jogo_rodando = True
301
302     # Laço de repetição while que continua até "jogo_rodando" ser False.
303     while jogo_rodando:
304
305         # Evoca uma função que exibe o estado do tabuleiro atual nesse caso.
306         exibir_estado(estado_atual)
307         # Imprime a geração.
308         print(f"Geração: {geracao}")

```

```

311     # Atribuição de variável. Verifica condições de parada "antes" de calcular o próximo estado. Supõe que não acabou.
312     jogo_terminou_por_condicao = False
313     if estado_anterior is not None:
314         # Chama a função que verifica as condições de fim.
315         terminou = verificar_fim_jogo(estado_atual, estado_anterior)
316         if terminou:
317             # Se isso for verdade, continua e o jogo acaba.
318             jogo_terminou_por_condicao = True
319
320     # Se o jogo terminou por morte ou estabilidade, para o loop principal
321     if jogo_terminou_por_condicao:
322         # Atribuição de variável. O jogo realmente acaba.
323         jogo_rodando = False
324     else:
325         # Se o jogo não terminou, espera a interação do usuário ou interrupção.
326         try:
327             # Entrada de dados para continuar o jogo ou interromper.
328             input("Pressione Enter para avançar para a próxima geração (ou Ctrl+C para sair)...")
329         except KeyboardInterrupt:
330             print("\nJogo interrompido pelo usuário.")
331             # Pausa para o usuário ler a mensagem antes da próxima atualização da tela.
332             time.sleep(1)
333         # Atribuição de variável. O jogo acaba por interrupção.
334         jogo_rodando = False
335
336     # Estrutura condicional que atualiza os estados se o jogo ainda estiver rodando (não foi interrompido e não acabou).
337     if jogo_rodando:
338         # Guarda o estado atual para comparação na continuação.
339         copia_estado_anterior = []
340         # Laço de repetição que percorre as linhas do "tabuleiro_atual".
341         for linha_original in estado_atual:
342             # Cria uma cópia da lista interna.
343             copia_linha = list(linha_original)
344             copia_estado_anterior.append(copia_linha)
345         # Atribui a cópia completa à variável estado_anterior.
346         estado_anterior = copia_estado_anterior
347
348         # Atribuição de variável. Evoca uma função que calcula o próximo estado.
349         estado_atual = calcular_proximo_estado(estado_atual)
350         # Contador que aumenta +1 a cada geração.
351         geracao += 1
352
353     # Imprime uma frase.
354     print("\n--- Jogo Encerrado ---")

```

Figura 11 e 12. Criação de uma função principal para o jogo e execução

Cria uma função principal para o jogo, organizando todas as funções criadas até agora é uma única função, chamada “main()”. Ela configura as variáveis iniciais (estado_anterior, estado_atual, geracao) com valores de “início” (None ou 0). Um primeiro loop while garante que o usuário selecione um modo de início ('manual' ou 'aleatorio'), chamando a função correspondente para definir o estado_atual. Depois o loop principal (while jogo_rodando) começa: a cada iteração, exibe o tabuleiro e a geração; chama verificar_fim_jogo para verificar se tudo deve parar. Se não, espera a entrada do usuário (Enter ou Ctrl+C), tratando a interrupção. Apenas se o jogo ainda estiver ativo (jogo_rodando for True), ele executa os passos para a próxima geração: copia o estado_atual para estado_anterior, calcula o novo estado_atual com calcular_proximo_estado, e incrementa geracao. O loop continua até que jogo_rodando se torne False. Depois imprime uma frase que finaliza o jogo.

```
355
356 # Garante que o código só será executado diretamente.
357 if __name__ == "__main__":
358     main()
```

Figura 13. Estrutura condicional para o main e evocação do mesmo.

O último passo garante que o código só será executado diretamente pelo script e não por importações como módulos, por exemplo.

2.3. Ordem de Codificação

Nesse segundo problema PBL, também passei por muitas dificuldades, principalmente em relação a questão de definir (criar) funções antes de utilizá-las e ainda continua sendo difícil. A complexidade do problema aumentou, mas pelo menos obtive algum aprendizado com isso. Dito isso, utilizei a seguinte ordem de codificação:

1. Estabeleci que deveria haver um símbolo para células mortas e vivas, que cada célula só pode estar morta ou viva, que deveria haver um input inicial das células vivas, que precisava usar modularização e que o tabuleiro deveria ser 10×10 , mas comecei com um 3×3 .
2. Pensei em como exibir esse “tabuleiro” e estabeleci “.” para célula morta e “V” para célula viva, exibindo basicamente com a função “print”.
3. Pensei em como armazenar e cheguei a conclusão de usar um sistema de booleanos onde uma célula viva é “True” e morta é “False” e usar listas para armazenar esses estados.

4. Pensei em como transferir o estado das células e basicamente era somente igualar um estado anterior e também aprender a como delimitar os vizinhos e calcular os mesmos.
5. Pensei em quando o jogo acabaria e estabeleci que terminaria quando todas as células estivessem mortas ou quando atingisse a “vida parada”.
6. Pensei em como começar o jogo e estabeleci que os valores seriam manualmente ou aleatoriamente definidos.
7. Utilizei modularização principalmente.

O “Jogo da Vida” foi produzido utilizando a linguagem de programação Python 3.13.2, com o Visual Studio Code 1.98 como Ambiente Integrado de Desenvolvimento (*IDE*) e o sistema operacional Windows 10 Home Single Language.

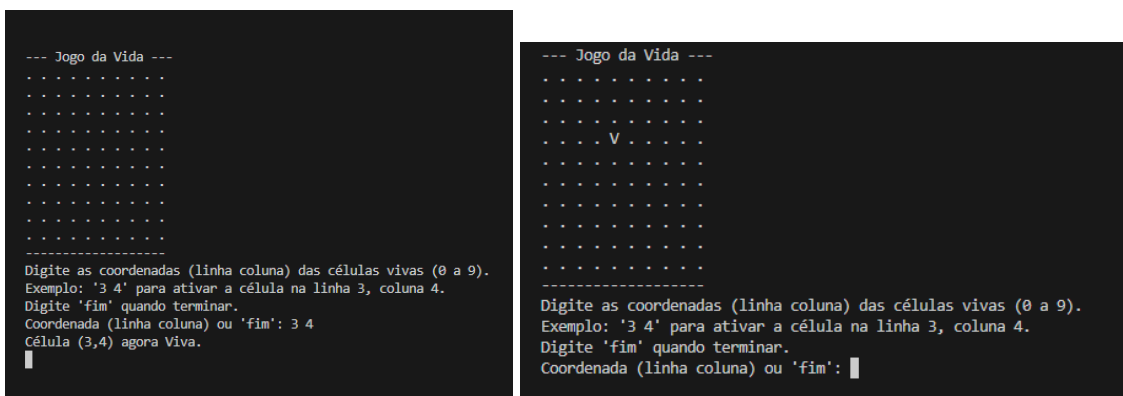
3. Resultados e Discussões

Basicamente, o programa pode ser dividido em inicialização e configuração, onde são importadas as bibliotecas e definidas as constantes, gerenciamento do tabuleiro, onde são criadas as funções de tabuleiro vazio e exibição do tabuleiro, lógica do jogo, onde é criada a função que verifica os vizinhos de uma célula determinada, modos de inicialização, onde funções são criadas para modos manual e aleatório, respectivamente e controle do fluxo do código, onde a função main ordena todas as funções criadas anteriormente.

Já em relação a utilização do programa, o usuário deve iniciar o programa e selecionar o modo de jogo escrevendo “manual” ou “aleatorio”. Caso o modo seja manual, um tabuleiro “morto” será exibido e o usuário será instruído a inserir as coordenadas no formato (linha coluna) e selecionará todas as células que ele quer tornar vivas, sendo que ele pode reverter isso com a mesma coordenada. Depois, ele deverá escrever “fim” para iniciar a simulação. Caso o modo seja aleatório, células vivas já estarão no tabuleiro. Em ambos os casos, o usuário estará com a simulação iniciada e basta clicar Enter para continuar ou Ctrl + C para interromper o jogo. Caso não interrompa, o jogo finaliza com todas as células mortas ou em “vida parada” em determinada geração.

Em relação às entradas do jogo, há a seleção do modo de jogo, a inicialização por coordenadas no modo manual, o avanço da simulação e o interrompimento da mesma. Quanto às saídas, há a exibição do tabuleiro, o contador de gerações, as mensagens de instrução, as mensagens de erro e as de fim de jogo.

3.1. Testes



```
--- Jogo da Vida ---
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
-----
Digite as coordenadas (linha coluna) das células vivas (0 a 9).
Exemplo: '3 4' para ativar a célula na linha 3, coluna 4.
Digite 'fim' quando terminar.
Coordenada (linha coluna) ou 'fim': 3 4
Célula (3,4) agora Viva.
█

--- Jogo da Vida ---
. . . . .
. . . . .
. . . . .
. . . V . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
-----
Digite as coordenadas (linha coluna) das células vivas (0 a 9).
Exemplo: '3 4' para ativar a célula na linha 3, coluna 4.
Digite 'fim' quando terminar.
Coordenada (linha coluna) ou 'fim': █
```

Figuras 1 e 2. Verificação de entrada das coordenadas.

Como pode ser visto, a funcionalidade de inserir coordenadas funciona corretamente.

```
Escolha o modo de início ('manual' ou 'aleatorio'): aleatorio

--- Jogo da Vida ---
V . . . V V . . . V
. . V . . V . . . .
V V . . V . . . . V
V . V V V V . . . V
. . V . . V V . . .
. V . . . . V V . .
. . V . . . V . . V
. . . . . V . . .
. . . . . V V . V .
V . V V V V V V V V
-----
Geração: 0
Pressione Enter para avançar para a próxima geração (ou Ctrl+C para sair)...
```

Figuras 3 e 4. Verificação de seleção de modo e funcionamento do modo aleatório.

Como pode ser visto, a funcionalidade de inserir o modo de jogo e de gerar caracteres aleatórios também funciona corretamente.

```
--- Jogo da Vida ---
. . . . V V . . . .
. . . . . V . . . .
V . . . . . . . . .
. . . . . . . . .
. . V V . . V V . .
. V V . . . V V V .
. . . . . V . V .
. . . . . V V V V
. . . V . . . V . V
. . . V V . . . V V
-----
Geração: 2
Pressione Enter para avançar para a próxima geração (ou Ctrl+C para sair)...
Jogo interrompido pelo usuário.

--- Jogo Encerrado ---
```

Figura 5. Verificação de Enter e de Ctrl + C.

Como pode ser visto, a funcionalidade de continuar e interromper a simulação funciona corretamente.



Figura 6 e 7. Verificação de finais de jogo.

Como pode ser visto, os dois finais ocorrem na simulação e funcionam corretamente.

4. Conclusão

Analisando todo o trabalho, acredito que a criação do “Jogo da Vida” foi bem-sucedida e todos os requisitos foram cumpridos, talvez com algumas funcionalidades “extras”, como a possibilidade de limpar a tela ou de evitar possíveis delays.

Além disso, como possíveis melhorias para o programa, poderiam ser desenvolvidas as seguintes funcionalidades: interface do programa, a possível seleção do tamanho do tabuleiro, um sistema de coordenadas por clique e uma melhor otimização do código.

4.1. Detalhe Importante

O programa foi criado com o auxílio de Inteligência Artificial (IA), mais especificamente o Google AI Studio, disponível em: <https://aistudio.google.com>.

5. Referências Bibliográficas

Lima, Leandro Zoucas de e Oliveira, Pedro Paulo Marques de. (2014) “ Jogo da Vida: Conceitos e Aplicações”, disponível em:

<https://bsi.uniriotec.br/wp-content/uploads/sites/31/2020/05/201406ZoucasMarques.pdf>;

Acesso em 03 de Junho de 2024